Graph Theory

Mid-Term Report

Submitted by

Teenu Anand

Summer Of Science

June 2026

Contents

1 Introduction **3**
- 1.1 Graph . . . 3
- 1.2 Loop . . . 3
- 1.3 Degree of vertex . . . 3
- 1.4 adjacency . . . 3

2 Types of Graphs **4**
- 2.1 Simple graph . . . 4
- 2.2 Directed graph . . . 4
- 2.3 Some common graphs . . . 4

3 Graph representations **5**
- 3.1 Adjacency Matrices . . . 5
- 3.2 Adjacency lists . . . 5
- 3.3 Representation in C++ . . . 5

4 Graph traversal methods and recursive graph exploration **6**
- 4.1 Breadth First Search(BFS) . . . 6
  - 4.1.1 How BFS Works . . . 6
  - 4.1.2 BFS C++ code . . . 6
- 4.2 Depth First Search(DFS) . . . 7
  - 4.2.1 How DFS Works . . . 7
  - 4.2.2 Recursive DFS code . . . 7

5 Connected Components **7**
- 5.1 Why BFS/DFS is Used . . . 7

6 Graph Traversal Trees **8**
- 6.1 BFS Traversal Tree . . . 8
- 6.2 DFS Traversal Tree . . . 8
- 6.3 Applications of Traversal Trees . . . 8

7 Bipartite Graphs **9**
- 7.1 Theorem . . . 9
  - 7.1.1 Proof . . . 9

8 Matching Problems **10**
- 8.1 Sex in America: An Application of Bipartite Graphs . . . 10
  - 8.1.1 Modeling the Problem . . . 10
  - 8.1.2 Degree of a Vertex . . . 10
  - 8.1.3 Key Observation . . . 10
  - 8.1.4 Average Number of Partners . . . 10
  - 8.1.5 Interpretation . . . 11

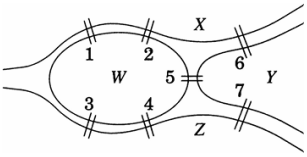
9 Hand Shaking Lemma **11**
- 9.1 Proof . . . 11
- 9.2 Bipartite Matchings . . . 11
  - 9.2.1 Matching in Bipartite Graphs . . . 11

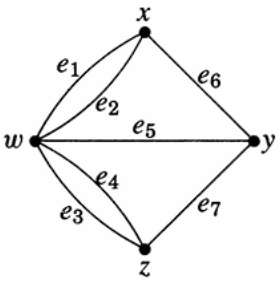
9.2.2 Hall's Matching Theorem 12
9.3 Hall's Matching Theorem 12
9.3.1 Bottlenecks 12
9.3.2 Degree-Constrained Bipartite Graphs 12
9.3.3 Regular Bipartite Graphs 13
9.3.4 Applications 13

10 Graph Coloring and Chromatic Number **13**
10.1 Graph Coloring 13
10.2 Chromatic Number 14
10.3 Examples 14
10.3.1 Complete Graph 14
10.3.2 Cycle Graphs 14
10.3.3 Bipartite Graphs 14
10.3.4 Trees 15
10.4 Important Results 15
10.5 Greedy Coloring Algorithm 15
10.6 Applications of Graph Coloring 15

1 Introduction

The birth of graph theory will suggest our basic definition of graph.
1.1.1. Example. *The Königsberg Bridge Problem.* The city of Königsberg was located on the Pregel river in Prussia. The city occupied two islands plus areas on both banks. These regions were linked by seven bridges as shown on the left below. The citizens wondered whether they could leave home, cross every bridge exactly once, and return home. The problem reduces to traversing the figure on the right, with heavy dots representing land masses and curves representing bridges.
The model on the right makes it easy to argue that the desired traversal does not exist.





Each time we enter and leave a land mass, we use two bridges ending at it. We can also pair the first bridge with the last bridge on the land mass where we begin and end. Thus existence of the desired traversal requires that each land mass be involved in an even number of bridges. This necessary condition did not hold in Königsberg. ■

1.1 Graph

A graph G is a triple consisting of vertex set V(G), an edge set E(G),and a relation that associates with each edge two vertices(not necessarily distinct) called its endpoints.

1.2 Loop

A loop is an edge whose endpoints are equal.
Multiple edges are edges having the same pair of endpoints.

1.3 Degree of vertex

, and an edge is said to be incident to the vertices it joins. The number of edges incident to a vertex v is called the degree of the vertex and is denoted by deg(v); equivalently, the degree of a vertex is equals the number of vertices adjacent to it.

1.4 adjacency

Two vertices in a simple graph are said to be adjacent if they are joined by an edge

2 Types of Graphs

2.1 Simple graph

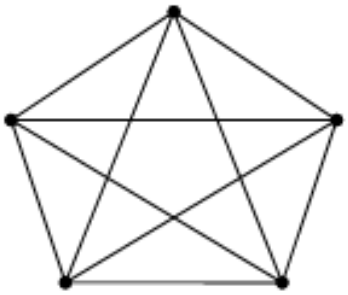
A simple graph is a graph having no loops or multiple edges
When u and v are endpoints of an edge,they are adjacent and are neighbors.

2.2 Directed graph

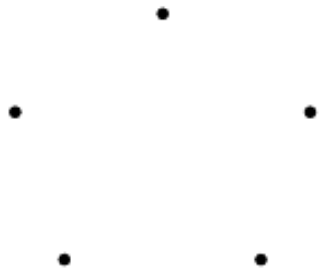
A directed graph (or digraph)(V,E) consists of a nonempty set of vertices V and a set of directed edges (or arcs) E. Each directed edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (u,v) is said to start at u and end at v.

2.3 Some common graphs

Some graphs come up so frequently that they have names. The complete graph on n vertices, denoted Kn, has an edge between every two vertices, for a total of $n \cdot n - 1/2 = edges$
The empty graph has no edges at all. The empty graph has no edges at all.



(a) Complete graph (K_5)



(b) Empty graph with 5 nodes

The n-node graph containing n-1 edges in sequence is known has the line graph L_n

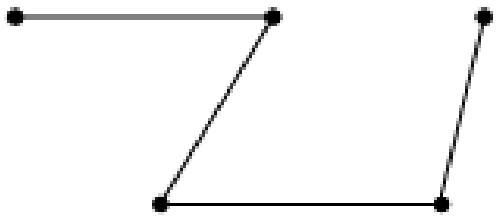


Figure 3: A 5-node line graph L_5

3 Graph representations

3.1 Adjacency Matrices

There are many ways to represent a graph.you can draw it or you can represent it with sets.another way is to represent it with adjacency matrix
Given an n-node graph $G = (V, E)$ where $V = \{v_1, v_2, ..., v_n\}$,the adjacency matrix for G is the $n \times n$ matrix $A_G = \{a_{ij}\}$ where

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E \\ 0 & \text{otherwise} \end{cases}$$

3.2 Adjacency lists

Specifies the vertices adjacent to each vertex of graph.

3.3 Representation in C++

Graph implementation in C++ using an adjacency list.

```
#include <iostream>
#include <vector>
#include <list>

using namespace std;

class Graph {
    int V;
    vector<vector<pair<int, int>>> adj;

public:
    Graph(int vertices) {
        V = vertices;
        adj.resize(V);
    }

    void addEdge(int u, int v, int weight = 1) {
        adj[u].push_back({v, weight});
        adj[v].push_back({u, weight});
    }
    void printGraph() {
        for (int i = 0; i < V; i++) {
            cout << "Vertex " << i << " -> ";

            for (auto edge : adj[i]) {
                cout << "(" << edge.first
                     << ", weight = "
                     << edge.second << ") ";
```

```
            }

            cout << endl;
        }
    }
};
```

4 Graph traversal methods and recursive graph exploration

4.1 Breadth First Search(BFS)

BFS is a graph traversal algorithm that visits nodes level by level starting from a source node.
It explores all immediate neighbors first, then neighbors of those neighbors, and so on.

4.1.1 How BFS Works

1. Put the starting node into queue
2. Mark it as visited.
3. Repeatedly:
    - Remove a node from the queue.
    - Visit all its unvisited neighbors. Add those neighbors to the queue.

4.1.2 BFS C++ code

```
void bfs(int start, vector<vector<int>>& adj) {
    vector<bool> visited(adj.size(), false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        cout << node << " ";

        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
```

```
    }
}
```

4.2 Depth First Search(DFS)

DFS is a graph traversal algorithm that explores a path as deeply as possible before backtracking.
Instead of visiting nodes level by level like BFS, DFS keeps moving to an unvisited neighbor until it can't go further.

4.2.1 How DFS Works

1. Visit the current node.
2. Mark it as visited.
3. Go to an unvisited neighbor.
4. Repeat until no unvisited neighbors remain.
5. Backtrack to the previous node and continue.

4.2.2 Recursive DFS code

```
    void dfs(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    cout << node << " ";

    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, adj, visited);
        }
    }
}
```

5 Connected Components

A connected component in an undirected graph is a group of vertices where every vertex can be reached from every other vertex in that group.

5.1 Why BFS/DFS is Used

1. Run DFS/BFS from current node.
2. Mark all reachable node.
3. Count one connected component
4. Repeat for remaining unvisited nodes

```
int countComponents(int n,
                    vector<vector<int>>& adj)
{
    vector<bool> visited(n, false);
    int count = 0;

    for(int i = 0; i < n; i++)
    {
        if(!visited[i])
        {
            dfs(i, adj, visited);
            count++;
        }
    }

    return count;
}
```

6 Graph Traversal Trees

A graph traversal tree is the tree formed by the edges used during a BFS or DFS traversal.

6.1 BFS Traversal Tree

A BFS traversal tree is constructed when Breadth-First Search is performed on a graph. Starting from a source vertex, BFS explores all neighboring vertices before moving to the next level of vertices. The edges used to reach previously unvisited vertices form the BFS traversal tree. This tree represents the shortest-path structure from the source vertex in an unweighted graph.

6.2 DFS Traversal Tree

A DFS traversal tree is constructed when Depth-First Search is performed on a graph. DFS explores as far as possible along each branch before backtracking. The edges used to discover new vertices form the DFS traversal tree. Unlike a BFS tree, a DFS tree does not necessarily represent shortest paths but illustrates the recursive exploration of the graph

6.3 Applications of Traversal Trees

- Finding connected components in a graph.
- Detecting cycles.
- Finding paths between vertices.
- Topological sorting of directed acyclic graphs
- Network analysis and routing.

- Constructing spanning trees.

7 Bipartite Graphs

A bipartite graph is a graph where the vertices can be split into two groups such that:

- Every edge connects a vertex from one group to a vertex in the other group.
- No edge exists between two vertices in the same group.

Equivalently, a graph is bipartite if it can be colored using two colors so that no two adjacent vertices share the same color.

7.1 Theorem

A graph is bipartite if and only if it contains no odd-length cycle.

7.1.1 Proof

We has to prove both directions.
1. If a graph is bipartite, then it has no odd cycle
Assume the graph is bipartite.
Let the two sets be:U and V.
Now suppose there is a cycle.as the graph is bipartite vertices on the cycle alternate between U and V.to return to initial vertex the number of edges traveled must be even.
Therefore every cycle in a bipartite graph has even length.
Hence odd cycles cannot exist.
2. If a graph has no odd cycle,then it is bipartite
Choose any vertex r.
Define:

- set U:vertices at even distance from r.
- set V:vertices at odd distance from r.

We claim every edge joins a vertex in U to one in V.
Suppose not.Assume there is an edge connecting two vertices in the same set.
Case 1:Both vertices are in U As both there distances fron r is even.r and those 2 vertices form a odd length cycle.
Hence Contradiction!
Case 2:Both vertices are in V Similarly,odd+odd+1 is also odd producing odd cycle.
Contradiction.
Therefore no edge can connect two vertices in the same set.
Hence every edge goes between U and V.

So the graph is bipartite.

8 Matching Problems

8.1 Sex in America: An Application of Bipartite Graphs

Graph theory can be applied to analyze real-world social networks. One interesting example is the study of opposite-gender sexual partnerships in the United States.

8.1.1 Modeling the Problem

Let $G = (V, E)$ be a graph where each vertex represents a person in America. The vertex set is divided into two disjoint sets:

- M = set of all males
- F = set of all females

An edge is drawn between a male and a female if they have been sexual partners. Since edges only connect vertices from different sets, G is a **bipartite graph**.

8.1.2 Degree of a Vertex

The degree of a vertex represents the number of opposite-gender partners of that person.

- Degree of a male vertex = number of female partners.
- Degree of a female vertex = number of male partners.

8.1.3 Key Observation

Every partnership corresponds to exactly one edge in the graph. Each edge is incident to one male vertex and one female vertex. Therefore, the sum of the degrees of all male vertices equals the sum of the degrees of all female vertices:

$$\sum_{x \in M} \deg(x) = \sum_{y \in F} \deg(y)$$

8.1.4 Average Number of Partners

The average number of opposite-gender partners for males is

$$\frac{\sum_{x \in M} \deg(x)}{|M|}$$

and the average number of opposite-gender partners for females is

$$\frac{\sum_{y \in F} \deg(y)}{|F|}.$$

Dividing the degree-sum equation by $|M||F|$, we obtain

$$\frac{\sum_{x \in M} \deg(x)}{|M|} \frac{|F|}{|M|} \cdot \frac{\sum_{y \in F} \deg(y)}{|F|}.$$

Thus,

$$\text{Average Male Partners} \frac{|F|}{|M|} \times \text{Average Female Partners}.$$

8.1.5 Interpretation

According to U.S. Census data, there are slightly more females than males. Hence,

$$\frac{|F|}{|M|} \approx 1.035.$$

Therefore, the average number of opposite-gender partners for males can only be about 3.5 % higher than that for females.

9 Hand Shaking Lemma

The sum of the degrees of the vertices in a graph equals twice the number of edges

9.1 Proof

Every edge contributes two to the sum of the degrees, one for each of its endpoints.

9.2 Bipartite Matchings

A bipartite graph is a graph whose vertices can be partitioned into two disjoint sets L and R such that every edge connects a vertex in L to a vertex in R.

A **bipartite graph** is a graph $G = (V, E)$ together with a partition of its vertex set into two subsets L and R such that every edge has one endpoint in L and the other endpoint in R.

Bipartite graphs are widely used to model relationships between two different types of objects, such as students and courses, workers and jobs, or men and women.

9.2.1 Matching in Bipartite Graphs

A matching is a set of edges that do not share any common vertex. In a bipartite graph, a matching represents a valid assignment between the two sets of vertices.

A **matching** in a graph is a set of edges such that no two edges have a common endpoint.

For example, in a graph representing men and women, a matching corresponds to pairing each man with a distinct woman.

A matching is said to **cover** a set of vertices L if every vertex in L is incident to an edge in the matching.

A **perfect matching** is a matching that covers every vertex of the graph.

9.2.2 Hall's Matching Theorem

A natural question is whether it is possible to find a matching that covers all vertices on one side of a bipartite graph.

To answer this question, Hall's Matching Theorem provides a necessary and sufficient condition.

For a subset $S \subseteq L$, the neighborhood of S, denoted by $N(S)$, is the set of all vertices in R adjacent to at least one vertex in S:

$$N(S) = r \in R \mid (s, r) \in E \text{ for some } s \in S.$$

9.3 Hall's Matching Theorem

Let G be a bipartite graph with vertex partition L and R. There exists a matching that covers L if and only if

$$|N(S)| \geq |S|$$

for every subset $S \subseteq L$.

This condition means that every group of vertices in L collectively has at least as many available neighbors in R as its size.

9.3.1 Bottlenecks

A subset $S \subseteq L$ is called a **bottleneck** if

$$|S| > |N(S)|.$$

If a bottleneck exists, then Hall's condition fails and a matching covering L cannot exist.

9.3.2 Degree-Constrained Bipartite Graphs

Checking Hall's condition for every subset can be difficult. A simpler sufficient condition is based on vertex degrees.

A bipartite graph with partition L, R is called **degree-constrained** if

$$\deg(l) \geq \deg(r)$$

for every $l \in L$ and $r \in R$.

If a bipartite graph G with partition L, R satisfies $|L| \leq |R|$ and is degree-constrained, then there exists a matching that covers L.

This theorem guarantees a matching without explicitly checking Hall's condition.

9.3.3 Regular Bipartite Graphs

A graph is called **regular** if every vertex has the same degree.

Examples include 2-regular graphs and 3-regular graphs, where every vertex has degree 2 or 3 respectively.

Every regular bipartite graph has a perfect matching.

Since all vertices have equal degree, the graph automatically satisfies the degree-constrained property. Therefore, Hall's theorem ensures the existence of a matching that covers all vertices, resulting in a perfect matching.

9.3.4 Applications

Bipartite matching has many practical applications:

- Job assignment problems
- Student-course allocation
- Resource allocation
- Network design
- Marriage and pairing problems
- Scheduling and task assignment

Thus, bipartite matchings provide an important connection between graph theory and real-world optimization problems.

10 Graph Coloring and Chromatic Number

Graph coloring is one of the most important topics in graph theory. It involves assigning colors to the vertices of a graph subject to certain constraints. Graph coloring has numerous applications in scheduling, resource allocation, map coloring, and network design.

10.1 Graph Coloring

Definition 1 *A **vertex coloring** of a graph $G = (V, E)$ is an assignment of colors to the vertices such that no two adjacent vertices receive the same color.*

A coloring satisfying this condition is called a **proper coloring**.

For example, if two vertices are connected by an edge, they must be assigned different colors.

10.2 Chromatic Number

Definition 2 *The **chromatic number** of a graph G, denoted by $\chi(G)$, is the minimum number of colors required to properly color the vertices of the graph.*

The chromatic number measures how difficult it is to color a graph while satisfying the coloring constraints.

10.3 Examples

10.3.1 Complete Graph

A complete graph K_n contains an edge between every pair of vertices.

Since every vertex is adjacent to every other vertex, each vertex must receive a different color.

Therefore,

$$\chi(K_n) = n.$$

For example,

$$\chi(K_4) = 4.$$

10.3.2 Cycle Graphs

The chromatic number of a cycle depends on its length.

- If the cycle has even length, then

$$\chi(C_n) = 2.$$

“

- If the cycle has odd length, then

$$\chi(C_n) = 3.$$

“

For example,

$$\chi(C_4) = 2, \qquad \chi(C_5) = 3.$$

10.3.3 Bipartite Graphs

A bipartite graph can be divided into two disjoint vertex sets such that every edge joins vertices from different sets.

Therefore, every bipartite graph can be colored using two colors.

Hence,

$$\chi(G) = 2$$

for every connected bipartite graph containing at least one edge.

10.3.4 Trees

A tree is an acyclic connected graph.

Every tree is bipartite, and therefore can be colored using two colors.

Thus,

$$\chi(T) = 2$$

for every tree with at least two vertices.

10.4 Important Results

Theorem 1 *A graph is bipartite if and only if it is 2-colorable.*

This theorem establishes a direct relationship between bipartite graphs and graph coloring.

Theorem 2 *A graph is bipartite if and only if it contains no odd cycle.*

Since odd cycles require three colors, the absence of odd cycles guarantees that two colors are sufficient.

10.5 Greedy Coloring Algorithm

One simple method for coloring a graph is the greedy coloring algorithm.

1. Consider the vertices one by one.
2. Assign the smallest available color that has not been used by any adjacent vertex.
3. Continue until all vertices are colored.

Although the greedy algorithm does not always produce an optimal coloring, it often provides a good approximation.

10.6 Applications of Graph Coloring

Graph coloring has many practical applications:

- Timetable scheduling
- Exam scheduling
- Register allocation in compilers
- Frequency assignment in wireless networks
- Map coloring
- Resource allocation problems